Discover Artificial Intelligence

RESEARCH Open Access

A knowledge graph-integrated recommendation method for college student career planning

Xiaojun Xu[1] and Hui Gao[2*]

*Correspondence:
Hui Gao
smaltila@sina.com
[1]Zhonghua Vocational College, Yunnan University of Finance and Economics, Kunming 650000, China
[2]School of Economics and Management, Yunnan Forestry Vocational and Technical College, Kunming 650000, China

Abstract

Facing the complex and diverse interests of college students, current career planning recommendation methods often suffer from low accuracy. To tackle this, we propose a novel knowledge graph-integrated recommendation model. Our approach begins by constructing a comprehensive career planning knowledge graph. We then project sparse user and item vectors into a dense latent space. Leveraging triplet information from the graph, we form project neighborhoods and employ a Graph Convolutional Network to adaptively aggregate neighbor vectors, deriving enriched representations for students and careers. A key innovation is the fusion of these graph-based representations with those from a generalized matrix factorization component. The concatenated vectors are fed into a fully connected layer to output the final recommendation score. Extensive comparative experiments on three real-world career planning datasets demonstrate the superiority of our model, showing significant improvements in recommendation accuracy of at least 2.04%, 2.26%, and 1.34%, respectively, over state-of-the-art methods.

Keywords Career planning recommendation, Knowledge graph, Artificial intelligence, Natural language processing, Graph convolutional network

1 Introduction

College students are an important force for the future development of the country and society, and their career choices are directly related to social progress [1]. The career situation faced by college students is becoming increasingly complex, and traditional career planning methods can no longer meet their personalized and precise demands [2]. The investigation of career planning among university students serves as a catalyst for the enrichment and deepening of theoretical frameworks in the field, providing novel perspectives and empirical evidence. At present, college students lack systematic and scientific guidance in the process of career planning. Hence, designing a career planning recommendation approach based on artificial intelligence to provide precise career planning suggestions is helpful for college students to make appropriate decisions in their career development [3, 4]. The rapid development of Internet technology, while

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

bringing convenience to people's lives, has also led to a rapid increase in online information volume, making it difficult for people to filter out information that meets their personalized needs from a large amount of information [5]. The career planning recommendation system has the function of efficient information filtering and is widely used in various online platforms. It can help college students accurately obtain their needs while also providing personalized services for college students [6–8].

The high accuracy of recommendation methods is closely related to the number of available ratings in the data set. If the rating matrix between college students and items is sparse to some extent, it will greatly affect the accuracy of the recommendation method, while also often being accompanied by cold start problems [9]. In solutions for these kinds of issues, knowledge graphs are drawing increasing attention because they can create a graph-structured network built jointly by university students and projects. Such structures lay a solid foundation for recommendations [10]. Generally speaking, knowledge-graph-based recommendations can be divided into two classes: path-based approaches [11–13] and embedding-based approaches [14, 15]. Despite the satisfactory recommendation performance of existing knowledge-aware methods, they are constrained by inherent limitations, primarily the weak interpretability of embedding-based models and the issue of error propagation in path-based approaches. Path-based methods [16] extract the association paths between college students and items, which utilize relationship information among entities and enhance the interpretability of recommendations. During the modeling phase for college student-item interactions, relevant paths between the user and item nodes are identified as propagation channels. Student preferences are then diffused as information flow along these designated paths within the graph [17]. Enumerating all paths between a node pair risks incorporating irrelevant ones, which can cause cascading errors in the propagation of student preferences. Path-based methods extract associative paths between users and items, leveraging relational information between entities and enhancing recommendation interpretability. However, when modeling interactions between given user-item pairs, paths between user nodes and item nodes are extracted as propagation paths, and user preferences propagate along these paths within the graph. Considering all paths between a given pair may involve irrelevant paths, this leads to erroneous propagation of user preferences. Using such paths for inference may introduce noise into user preferences, thereby reducing recommendation accuracy.

In embedding-based approaches, student preferences are typically derived from the latent representations of users and items, which are mapped to corresponding preference scores [18]. Specifically, recently more and more work has begun to utilize Graph Convolutional Networks (GCNs), as GCNs can effectively handle non-regular structured data, etc [19, 20]. Zhu et al. [21] used RippleNet combined with Word2vec and a Factorization Machine (FM) model to build a recommendation model based on the fusion of college student-career planning knowledge interaction data. They dynamically fused two models, and solved the model using stochastic gradient descent, obtaining a more accurate model than typical recommendation algorithms. Zhang et al. [22] proposed the NGCF model, which uses traditional GCN to model higher-order association information between college students and career planning knowledge, obtaining more effective representations of college students and items to improve the model's recommendation performance; however, this model is complex and has a very high time

complexity. Ramazanova et al. [23] proposed applying GCN to a bipartite graph of interactions between college students and career planning knowledge, transforming the task of matrix completion in recommendations into link prediction on graphs, effectively solving the cold start problem in recommendation systems. Wang et al. [24] proposed a lightweight GCN-based recommendation model based on knowledge graph. It samples from entity neighbors as its receptive field, and through multiple iterations of embedding propagation in knowledge graphs, it acquires higher-order neighborhood information to calculate entity representations by combining the receptive field with neighborhood information and potential biases. Zha et al. [25] used a GCN-based architecture to learn college student demand intentions based on career planning knowledge awareness. Tatchum et al. [26] calculated embeddings for college students and career planning knowledge through propagation and aggregation in heterogeneous graphs.

The aforementioned GCN-based recommendation approaches have been demonstrated to successfully model higher-order relationships within knowledge graphs to deliver improved recommendations. Current state-of-the-art outcome on public datasets are predominantly attributed to the application of GCN-based architectures. However, these GCN-based approaches also exhibit certain limitations [27, 28]. They calculate preference scores solely from user and item embeddings, lacking much critical contextual information about users. Furthermore, deep learning-based recommendation algorithms focus on mining implicit features within the data, while entities and relationships in knowledge graphs are not entirely comprehensive, containing incomplete and missing information. This implies that in knowledge graph-based recommendation systems, certain entities or relationships may not be accurately captured, thereby affecting recommendation accuracy. Traditional recommendation models exhibit certain limitations when confronting issues such as data sparsity and user cold start. Knowledge graphs provide rich relational and semantic information, while technologies like GNNs can effectively extract item features from KGs. This has prompted increasing numbers of researchers to explore integrating knowledge graphs with recommendation systems to meet personalized user needs and address information overload. Consequently, these emerging technologies present new opportunities and challenges for recommendation systems, driving further refinement and innovation. This paper proposes a knowledge graph-integrated career planning recommendation method for college students. This method integrates the knowledge graph with matrix factorization to achieve the comprehensive utilization of the structural information of the knowledge graph and the interaction information of college students' career planning knowledge in the recommendation system, thereby enriching the feature representation, improving the accuracy and generalization ability of recommendations. It shows advantages in dealing with sparse data and cold start scenarios. The main contributions of this research are summarized in the following four aspects.

(1) Collect data on college students' career planning and extract information from structured career planning data to achieve the construction of a knowledge graph for college students' career planning.
(2) The integration of knowledge graphs and matrix factorization can achieve the comprehensive utilization of interactive information between college students and career planning knowledge, thereby improving the accuracy of recommendations.

(3) Capture global information through GCN aggregation. The feature update of each layer includes the neighbor information of the previous layer, making full use of the connection information between nodes, thereby achieving effective aggregation and representation learning of node features.
(4) Experiments were conducted on multiple real datasets to obtain the optimal number of neighbors and the number of sampling layers. Based on the obtained optimal parameters, comparative experiments were conducted. The results show that the proposed model improves the recommendation accuracy and real-time performance.

2 Construction of a knowledge graph for college students' career planning

2.1 Data collection for college student career planning

The creation of a knowledge graph for college students' career planning requires a systematic review of knowledge. Data collection is the foundation for building a college student career planning knowledge graph [29]. The data collection phase includes collecting self-assessment texts and knowledge base texts; self-assessment texts include students' professional backgrounds, career interest tendencies, career abilities, career values, and other data. Knowledge base texts come from sources such as national occupational standards, academic papers, industry analysis reports, job requirements, or literature and industrial environmental data suitable for building a knowledge base. The collection results may be in various text formats, which need to be converted into plain text during the data collection stage.

The collected plain text is unstructured data and needs natural language processing, splitting paragraphs into words while analyzing lexical grammatical attributes and labeling the attributes of each word. To annotate these attribute labels for vocabulary, it is necessary to establish an annotated dataset in advance, including part-of-speech tagging, named entity recognition (proper name *Nh*, location *Ns*, organization *Ni*), semantic role types (actor *ARG*0, recipient *ARG*1, purpose *PRP*, scope *EXT*), dependency syntactic relations (subject-verb relation *SBV*, object-verb relation *VOB*, prepositional object *FOB*, modifier-head relation *ATT*), and semantic dependency annotations.

Employing a context-aware relation extraction model, ambiguous relationships are clarified by leveraging contextual information from unstructured text. For instance, regarding "Students who have participated in e-commerce projects are suitable for internet-related careers," contextual analysis extracts features such as "Project Type = E-commerce" and "Career Field = Internet," clarifying the semantic relationship "Student-Suitable-Internet Career" rather than the ambiguous "Related." For "Position requires marketing experience," by integrating the industry sector and job responsibilities, "marketing experience" is refined into the precise relationship "Possesses-Marketing Experience in the FMCG Industry."

Establish consistency rules for career planning domains to validate information extracted from unstructured text. Examples include: "Skill requirements for the same occupation must align with industry-standard norms," "Career levels (entry-level/mid-level/senior) must correspond to skill requirement tiers," and "Majors and compatible occupations must comply with the Ministry of Education's major-to-industry mapping catalog." Use a rule engine to filter inconsistent information (e.g., "Entry-level algorithm engineers must be proficient in deep learning frameworks"—statements deviating from

industry norms). Trigger manual review for suspected conflicts (e.g., "Marketing majors are suitable for finance industry roles") and validate reasonableness through industry case studies.

By labeling vocabulary with attribute tags, it facilitates the implementation of more flexible clustering methods and more accurate entity relationship extraction in subsequent steps. For example, for text content in the career planning knowledge base: "Enterprises need new technologies to enhance competitiveness; software engineers can only achieve greater growth potential by continuously learning and researching new technologies, thus creating value for enterprises." First, the sentence is split into words, yielding terms such as "enterprise," "need," "new," and "technology." Next, each word's grammatical attributes are classified and labeled accordingly. For instance, "enterprise" and "technology" are nouns (tag *n*), "need" is a verb (tag *v*), and "new" is an adjective (tag *a*), as shown in Fig. 1. Students interested in backend development may also be drawn to database development, relevant programming languages, artificial intelligence, and big data development. The fundamental logic of the algorithm is to use items of interest to the user as an initial seed. Then, leveraging the pattern of wave propagation, it jumps outward in concentric circles across the knowledge graph, spreading from the seed items to other items. In other words, when modeling user preferences, the algorithm not only incorporates items within the seed's immediate scope into the association range but also includes other items associated with those initial items. After natural language processing, unstructured text data generates structured data including part-of-speech, semantics, and syntactic relations, making it easier to perform clustering analysis and providing entities, attributes, and relationship data for knowledge graph construction.

2.2 Establishing a knowledge graph for college students' career planning

Perform information extraction on the structured data generated during natural language processing to construct a college students' career planning knowledge graph.

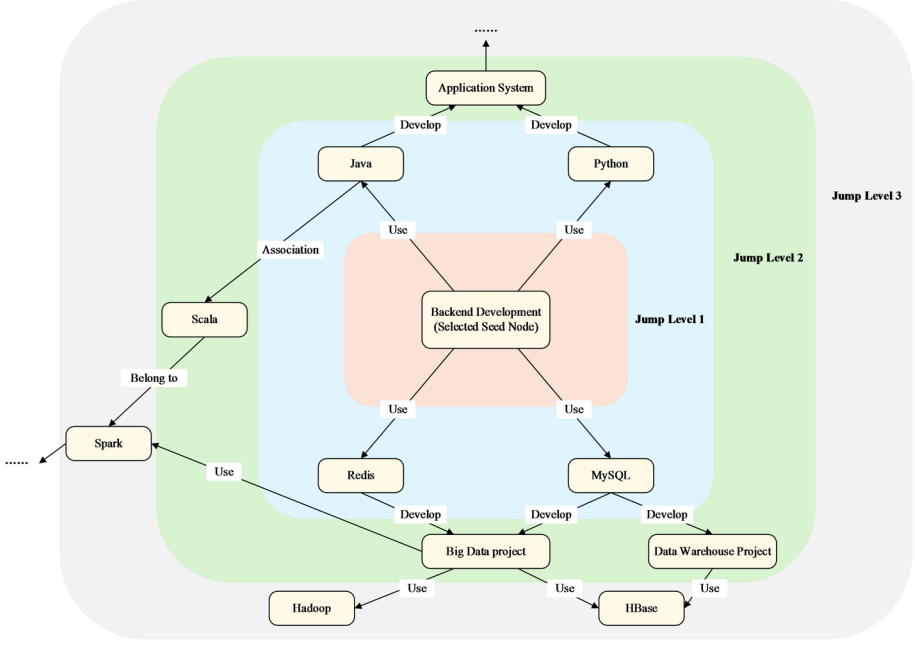


Fig. 1 Knowledge graph-based interest ripple diagram

Triplets are the most fundamental components of a knowledge graph, with a triplet data format such as "(Entity, Relation, Entity)" or "(Entity, Attribute, Attribute Value)." By extracting entities, relations, and attributes from vocabulary and attribute annotations obtained in the natural language processing stage, we form triplet data for building the knowledge graph, stored in the Neo4j graph database to realize the transformation of unstructured data into structured data. Taking the sentence in Fig. 2 as an example, entity relationships are extracted as: ['Enterprise', 'Needs', 'New Technology'], ['New Technology', 'Improves', 'Competitiveness'] and so on. It is evident that the extracted entity relationship results clearly express direct and indirect relationships between entities, forming structured knowledge resources.

Due to issues such as incomplete sentence components and imprecise language logic in the text, the constructed triplet data may suffer from missing components, leading to an incomplete knowledge graph. Both entities and relationships may be missing, so it is necessary to complete the knowledge graph. Knowledge representation learning is the most commonly used method for completing knowledge graphs [this sentence seems awkward], with a classic model being the TransE model [30]. During the process of knowledge completion, triplets in the graph need to be converted into embedding vectors, i.e., $(h, r, t) \rightarrow (\bar{h}, \bar{r}, \bar{t})$. In practice, word2vec [31] can be applied to achieve vector conversion. For correct triplets (h, r, t), they will satisfy a vector relationship $\bar{h} + \bar{r} = \bar{t}$, meaning that t is obtained by translating h through the relation r. Using the TransE model, the head and tail entities in the knowledge graph along with their relationships are encoded. If an entity or relationship is missing, $\bar{h} + \bar{r} = \bar{t}$ cannot be satisfied. By continuously adjusting the vectors of h, r, and t to make them as close as possible to satisfy $\bar{h} + \bar{r} = \bar{t}$, knowledge graph completion can be realized through word2vec+TransE modeling training. This helps reduce the impact of imprecise language logic on knowledge graph construction and improves its completeness.

The subsequent implementation of the college student career planning recommendation algorithm in the text is based on user preferences and the constructed knowledge base for content recommendations. The algorithm assumes that if a college student (user) is interested in something (item), other related items (items) are also potential

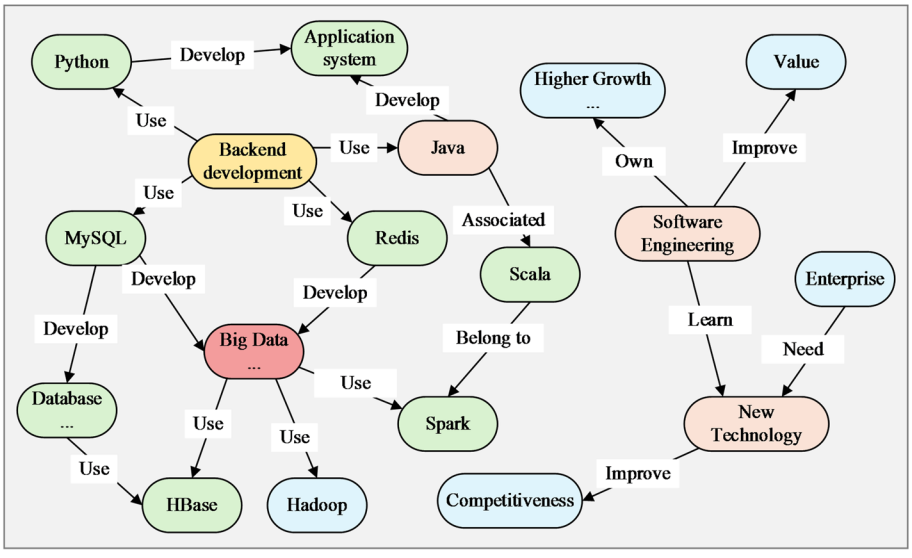


Fig. 2 Knowledge graph construction effectiveness

points of interest for the student. For example, with a career skill potential knowledge base recommendation, if a student is interested in backend development, they may also be interested in database development, relevant programming languages, artificial intelligence, big data development, etc.

For recommendations based on the career planning knowledge base data, after clustering processing of college students' points of interest, neo4j graph database is used as a persistent storage for the knowledge graph. The career planning knowledge base text acquired through data collection undergoes natural language processing and information extraction to construct triplet knowledge graph data, which is then stored in the neo4j database. The effectiveness of the knowledge graph construction can be checked using the database client tool, as shown in Fig. 2. The neo4j graph database supports setting different label classifications for storing data, and its accompanying client tools allow different node colors to be set for distinct label categories, making it easier to distinguish between different datasets.

3 Knowledge graph-based career planning recommendation method for college students

3.1 Overall model architecture

Traditional recommendation methods based on content or collaborative filtering have limited effectiveness in the face of information overload and the complex and diverse interests of college students [32]. To improve recommendation accuracy, researchers at home and abroad have begun introducing deep learning into recommendation algorithms. Deep recommendation algorithms can automatically learn feature representations from data, extract high-order features to provide richer semantic information and entity ass ociations, thereby better capturing the complex relationships between college students and items [33]. Although deep recommendations offer many advantages, cold starts occur when data is sparse, and the interpretability of recommendation results remains poor. To address these challenges, this paper proposes a college student career planning recommendation method integrated with knowledge graphs called KGIMCS. In KGIMCS, there exists a college student dataset $U = \{u_1, u_2, u_3, ..., u_m\}$ and a career planning knowledge dataset $V = \{v_1, v_2, v_3, ..., v_n\}$, where m represents the number of college students, and n represents the number of career planning knowledge items. The model framework is shown in Fig. 3, The model's primary functions are as follows.

(1) Construct the triplet of college student career planning knowledge graph using the method from the previous section to form tripartite graphical structure information. Convert the triplet information from the constructed knowledge graph into npy format to calculate the number of items and relationships. The triplet information in the file exists in a graph structure format, with the structure (head entity, relationship, tail entity). Build a dictionary by using the head entity and tail entity as keys respectively.

(2) Aggregate the knowledge graph triplet information. A dictionary is created for each entity, where the key value is the entity and the values are all entities associated with it. The values can serve as neighbors of the key. A GCN aggregator randomly selects k neighboring vector representations to aggregate into its own vector representation, calculating student vectors and relational vectors that represent students' preference relationships between different items. Deep learning yields the final college student vector representation and item vector representation.

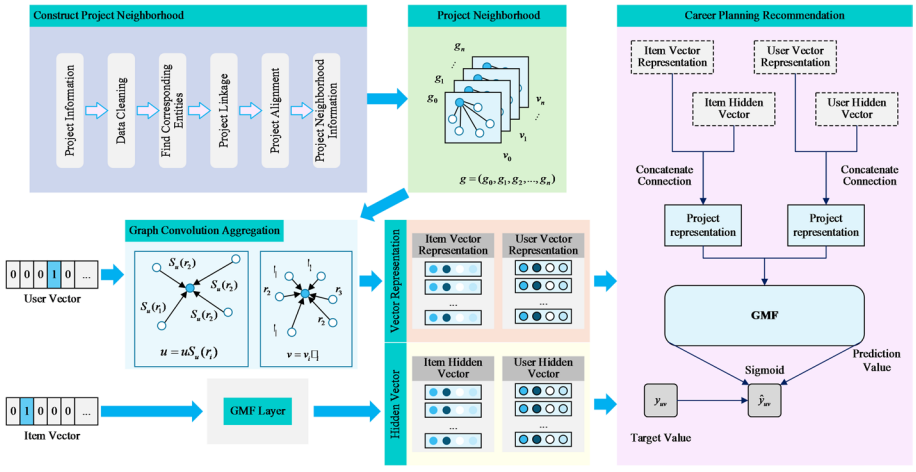


Fig. 3 Structure of a knowledge graph-based career planning recommendation model

Table 1 Symbol legend for the KGIMCS model

Symbol	Description
$s_u(r)$	The importance of relationship *r* to user u
$N_{(v)}$	Entity sets directly connected to career planning knowledge *v*
$H_{(v)}$	The receptive field of entity *v*
$\phi_{(u,\nu)}$	Final computational function for college student users and projects
agg	Entity neighborhood aggregation function
p_u	User latent vector
q_v	Projection vector
$\hat{y}_{uv}$	Score prediction function

(3) Concatenate the latent vectors of college students and items obtained from matrix decomposition with those derived from knowledge graph convolution aggregation and input them into a fully connected layer with a sigmoid as the last activation function to output a recommendation value between 0 and 1. In the KGIMCS recommendation model, for easier formula understanding, Table 1 provides explanations of the symbols.

3.2 GCN aggregates knowledge graph information

In the process of aggregating neighborhood career planning knowledge, the primary task is to convert different career planning knowledge graphs into personalized weighted graphs that reflect college students' preferences, thereby measuring the importance of relationships to college students. The purpose of this operation is to capture differences in attention levels for different relationships between college students and entities. For example, one student may be interested in computer-related career planning, while another student may be interested in news media-related career planning. Each student's interest level in different relationships varies; therefore, the mean value through multiplication is used to reflect this difference. The college student relationship function $s_u(r)$ is shown in Eq. (1).

$$s_u(r) = g(u, r) \tag{1}$$

where $u \in R^d$ stands for the characteristic vector of a college student, and $r \in R^d$ represents the feature vector for relationship type *r*. *d* indicates the vector dimension, and *g*

denotes the inner product function. Intuitively, $s_u(r)$ describes the importance of relationship *r* to college student *u*.

To represent the topological neighborhood structure of career planning knowledge *v*, this paper calculates the linear combination in the vicinity of *v* using the following Eq.

$$v^u_{N_{(v)}} = \sum_{e \in N_{(v)}} \tilde{s}_u(r) e \tag{2}$$

where $N_{(v)}$ denotes the set of entities linked to *v*, *e* is an entity, and $\tilde{s}_u(r)$ represents the normalized college student relationship score as shown in Eq. (3). When calculating the neighborhood representation of an entity, the college student relationship score acts as a personalized filter that aggregates neighborhoods based on deviations specific to each student's scores.

$$\tilde{s}_u(r) = \frac{exp^{f()}(s_u(r))}{\Sigma_{e \in N_{(v)}} exp^{f()}(s_u(r))} \tag{3}$$

In the constructed knowledge graph, the size of $N_{(v)}$ may vary significantly for each entity. To maintain a fixed and more efficient computational pattern for each batch, a set of neighbors of fixed size is uniformly sampled for each entity instead of using its complete neighborhood. Specifically, the neighborhood representation of an entity *v* is calculated as $H_{(v)}$, as shown in Eq. (4).

$$H_{(v)} \triangleq \{e \,|e \sim N_{(v)}\}, |H_{(v)}| = k \tag{4}$$

where *k* is a configurable constant that denotes the number of neighbors to be aggregated, and $H_{(v)}$ is referred to as the single-layer receptive field of entity *v*, namely knowledge about neighbors directly related to *v*. Figure 4 provides an example of two-layer receptive fields for a given entity, where *k* is set to 2. A single GCN layer computes the final representation of an entity by aggregating its own vector representation and those of neighboring vectors in its vicinity. In the figure, the green ellipse on the right represents the neighborhood representation, while the purple ellipse represents the representation after aggregation for a particular project, with h indicating the depth of the receptive field. By extending this single-layer aggregation process naturally to multiple layers, the model can explore college students' latent interests more broadly and deeply.

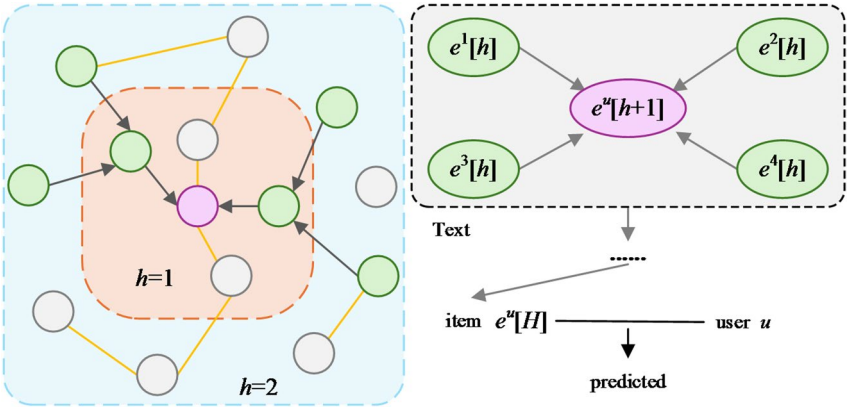


Fig. 4 Neighborhood distribution map with sampling depth 2

Using a single knowledge graph convolutional layer, an entity's final representation depends on itself and its adjacent entities, which is referred to as first-order entity vector representation. This process can be repeated by further propagating and aggregating the first-order representations in order to obtain second-order vector representations. Typically, an entity's first-order vector representation is a composite of its initial embedding and the embeddings of its neighbors within the *h* hop neighborhood.

The knowledge graph convolutional algorithm is shown in Table 2. The designed knowledge graph convolutional algorithm traverses all possible college student career planning knowledge pairs (u, v). It first iteratively computes the individual field H of v level by level, then repeats the aggregation process for H times. In each epoch H, it computes the neighborhood representation of each entity, aggregates it with its own vector representation, and uses this result in the next iteration. Ultimately, the H-th order entity representation is v^u. Equation (5) uses ϕ_1 to denote the final embedded vector representing the college student's learning behavior and career planning knowledge.

$$\phi_1(u, v) = (u, v^u) \tag{5}$$

The last step in knowledge graph convolution is aggregating v and its neighborhood representation $H_{(v)}$ into an individual vector. Since different aggregation methods have varying effects on the results, this chapter selects three graph convolutional aggregators for experimental comparison and analysis, represented as $agg : R^d \times R^d \rightarrow R^d$.

The first is the Sum aggregator. It sums two representation vectors and then applies nonlinear transformation. The equation is given below, where W and b are linear transformation weights and bias respectively, and σ stands for the nonlinear function Sigmoid.

$$agg_{sum} = \sigma(W \times (v + v^u H_{(v)}) + b) \tag{6}$$

The second is the Concat aggregator. It directly uses the neighbor representations of entity v as its output representation. The corresponding equation is shown below.

Table 2 Knowledge graph convolution algorithm process

Algorithm 1: Knowledge graph convolution process

Input: College student interaction matrix Y, knowledge graph triplet information G, parameter H, college student relationship score $\tilde{s}_u(r)$

Output: Embedded vector representations of college students and career planning

Initialize global model w_g, synthetic data $\hat{D}_g$, training data D_t, and robust ensemble R.

1. **for** (u, v) in Y **do**:
2. Obtain the neighborhood of $item_v$;
3. $e^u[0] \leftarrow e, \forall e \in M[0]$
// Iterated H times
4. **end**
5. **for** h = 1 to H **do**:
6. **for** $e \in N[h]$ **do**:
// Compute neighborhood representation
7. $e^u_{S(e)}[h-1] \leftarrow \sum_{e' \in S(e)} \tilde{s}_u(r) e'^u[h-1]$;
// Aggregate self-representation and neighborhood representation
8. $e^u[h] \leftarrow agg(e^u_{S(e)}[h-1], e^u[h-1])$
//Acquire the final career planning knowledge embedding representation
9. $v^u, u \leftarrow e^u[H]$
10. **end**

$$agg_{concat} = \sigma(W \times concat(v, v^u H_{(v)}) + b) \tag{7}$$

The third is the Neighbour aggregator. It directly takes the collection information of user neighbors as the user vector representation.

$$agg_{neighbor} = \sigma(W \times (v^u H_{(v)}) + b) \tag{8}$$

In knowledge graph convolutional aggregation, aggregation is considered a core step because the identification of career planning knowledge depends on aggregating it with adjacent items. This paper will elaborate further on this in the experimental section.

3.3 Matrix factorization knowledge information for college students and career planning

Matrix factorization (MF) performs collaborative filtering using inner products, where college students and career planning knowledge are mapped to latent feature vectors [34]. The MF model models the latent factors of college students and career planning knowledge in a bidirectional interactive manner. It assumes that each dimension in the latent space is independent and linearly combines them with equal weights to form a linear model of the latent vector [35]. Such design limits the expressive power of MF, preventing it from fully capturing the complexity of interactions between college students and career planning knowledge. In MF, college students and career planning knowledge are mapped into the same latent space. The similarity between two college students can be measured by an inner product. Choosing a higher-dimensional embedding vector to address this issue may lead to model overfitting, thereby affecting the generalization performance of the model [36].

Traditional MF models predict interactions solely through latent vector inner products, failing to capture complex patterns. Neural MF incurs high computational costs, requires substantial data to prevent overfitting, and suffers from poor model interpretability. In contrast, the proposed Generalized Matrix Factorization (GMF) overcomes the linear limitations of traditional MF, captures intricate interaction patterns, and achieves efficient computation through nonlinear activation and a simple interaction layer, making it suitable for resource-constrained scenarios. This paper designs the GMF used to model interactions between college students and career planning knowledge. It improves upon the traditional MF model by introducing nonlinear activation functions and learning weights. The interaction between college students and career planning knowledge is set as an element-wise product of latent feature vectors. Its overall framework is shown in Fig. 5.

The GMF model uses the Sigmoid activation function to perform dot product operations on the latent feature vectors of college students and career planning knowledge, then combines the results with learned weights to generate the final prediction result. This nonlinear combination approach enables the GMF model to better capture the nonlinear relationships between college students and career planning knowledge, enhancing the model's expressive power and recommendation accuracy. Let the hidden vector of a college student be p_u, and the hidden vector of an item be q_v. Then its mapping function is shown in Eq. (9), where $\odot$ represents the element-wise product of vectors.

$$\phi_2(u, v) = p_u \odot q_v \tag{9}$$

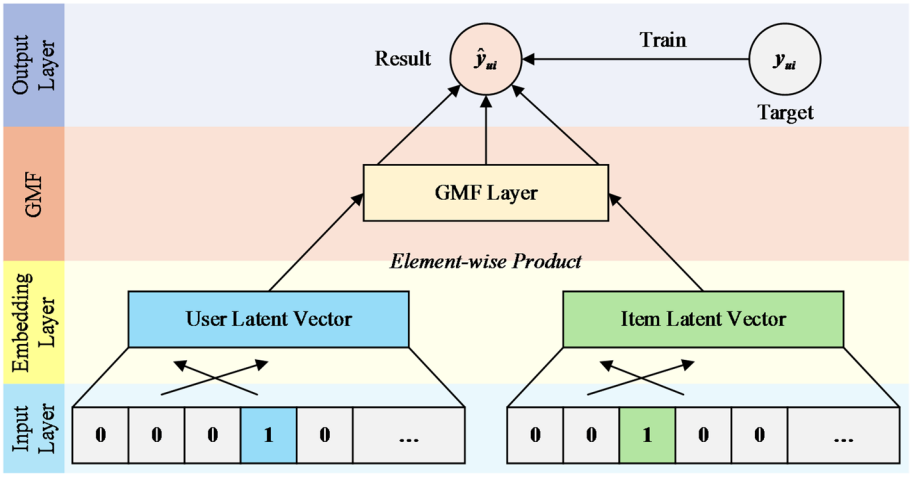


Fig. 5 GMF model

3.4 Career planning recommendation based on knowledge graph and matrix factorization

After obtaining the complete latent feature vectors for college students and career planning knowledge, GMF adopts a linear kernel to model the latent characteristic interactions between college students and career planning knowledge. The knowledge graph convolution utilizes a nonlinear kernel to learn interaction functions between college students and career planning knowledge. To effectively integrate vector information derived from GMF and knowledge graph convolution so that they can reinforce each other and learn complex user-item interactions, simply let GMF and knowledge graph convolution first learn individual embeddings separately. The cross-fusion of GMF and GCN outputs is not a simple concatenation but achieves feature complementarity through weighted combination, as shown in Eq. (10). This fusion approach significantly outperforms single-source representations in capturing complex relationships, enhancing feature robustness, and improving task performance.

$$\phi' = \alpha\phi_{GMF} + \beta\phi_{KGCN} \tag{10}$$

$$\hat{y}_{uv} = \sigma(\phi') \tag{11}$$

where α and β are weighting coefficients, $\alpha + \beta = 1$, ϕ_{GMF} represents the college student vector and career planning knowledge vector obtained through generalized matrix factorization, while ϕ_{KGCN} indicates the college student vector and career planning knowledge vector aggregated by graph convolution to extract information from the knowledge graph, ϕ' is the result of the fusion of ϕ_{GMF} and ϕ_{KGCN}. The two parts of vectors are concatenated to obtain the final college student vector and career planning knowledge vector. $\hat{y}_{uv}$ is calculated as the dot product between the college student vector and the career planning knowledge vector, resulting in a numerical value within [0,1], representing the final recommendation result.

To improve computational efficiency, negative sampling strategies are used during training. Its loss function is shown in Eq. (12).

$$L = \sum_{u \in U} (\Sigma_{v:y_{uv}=1} J(y_{uv}, \hat{y}_{uv}) - \sum_{i=1}^{T^u} E_{\text{ui}} \sim P(v_i) J(y_{uv}, \hat{y}_{uv})) + \lambda ||F||_2^2 \tag{12}$$

where y_{uv} and $\hat{y}_{uv}$ represent the true results and predicted results respectively, J is the cross-entropy loss, P is the set of negative samples, T^u denotes the number of negative samples, $T^u = |\{v = y_{uv} = 1\}|$ follows a uniform distribution, and the last term is an L2 regularizer.

4 Experimental and results and analysis

4.1 Experimental setting and comparative methods

4.1.1 Dataset and experimental environment

The experimental environment of this article is the win11 64-bit operating system with 32G memory, and the programming software used is Pycharm2021, with Python3.8 as the programming language and Tensorflow2.3 as the open-source machine learning framework used. In the experiments, the benchmark model employed the same experimental environment, parameter settings, and datasets as the proposed model. In this paper, the following datasets were selected for experimentation: TF, a career planning dataset for university students from a Chinese university; JD, a career planning dataset for university students from a major recruitment website; and TREC, a publicly released student career planning dataset from the University of Minnesota. The detailed information of the datasets is shown in Table 3. TF contains 1,409 college students, 5,637 career planning knowledge items, and 26,975 interaction records. JD includes 6,036 college students, 2,445 career planning knowledge items, and 123,772 interaction records. TREC contains 162,036 explicit ratings (ranging from 0 to 10) across 15,947 projects submitted by 20,860 university students.

4.1.2 Parameter settings

In the experiment, the dimension size of the embedding vectors on the three datasets was all set to 15, Adam was used as the model optimizer, the learning rate was set to 0.001, and the neural network was set to 5 layers, with 128 neurons in each layer. For the sampling depth h and the number of aggregated neighbors k, the impact on the model parameters will be experimentally analyzed in the subsequent sections of this paper. Therefore, in the following table, the parameters of h and k are temporarily set to 2 and 8 respectively. The other parameter Settings in the proposed recommendation model and the benchmark model are shown in Table 4.

4.1.3 Comparative methods

In this subsection, we compare the proposed model KGIMCS with related models LFM [21], NGCF [22], KGMR [23], IHGCN [24], CMAKG [25], KGLRS [26], MVIDL [37], MACPF [38], AMCSI [39], JFormer [40] through experiments to verify the effectiveness of KGIMCS.

Table 3 Detailed information about the dataset

Data description	TF	JD	TREC
College students	1409	6036	20,860
Knowledge of career planning	5637	2445	15,947
Interactions	26,975	123,772	162,036
Entitles	8263	6011	10,398
Relations	59	11	21
KG triples	10,552	136,928	52,379

Table 4 Parameter settings

Parameter	Parameter description	TF	JD	TREC
k	Aggregated neighbor count	8	8	8
Dim	Dimensions of student and entity embeddings	8	16	32
h	Sampling depth	2	2	2
Lr	Learning rate	5×10^{-4}	2×10^{-5}	2×10^{-5}
Epochs	Number of iterations	15,518	1,241,995	60,787

(1) LFM: This algorithm is based on the idea of matrix factorization, decomposing the college student-career planning knowledge interaction matrix into two low-dimensional latent feature matrices representing the latent interests of college students and the latent attributes of items. The core idea of the LFM algorithm is to predict college students' ratings or preferences for career planning knowledge by learning potential feature vectors of college students and career planning knowledge.

(2) NGCF: This algorithm is a hybrid recommendation model that combines factorization machines (FM) with GCN, aiming to improve the performance of recommender systems. It models low-order cross features through FM models and uses GCN to learn high-order feature crossings, combining outputs from both for final prediction results.

(3) KGMR: This algorithm is a recommendation algorithm based on GCN that combines tensor decomposition with neural network recommendation algorithms using MLP models.

(4) IHGCN: This algorithm integrates collaborative filtering and knowledge graph technologies. The core idea of the IHGCN algorithm is to jointly model interaction data between college students and career planning knowledge as well as entity and relationship information in knowledge graphs.

(5) CMAKG: This algorithm combines the information of entities and relationships in knowledge graphs as well as interactive behaviors between college students and career planning knowledge using GCN to make more accurate recommendations and learn college students' interests.

(6) KGLRS: Calculates embeddings for college students and career planning knowledge through propagation and aggregation in a heterogeneous graph.

(7) MVIDL: This method achieves career planning recommendations through multi-view learning. It designs a multi-task learning framework enabling different view models to share underlying feature representation. such as user skill features that can be simultaneously applied to job matching and path recommendations.

(8) MACPF: This method designs a hybrid recommendation system based on meta-heuristic algorithms and collaborative filtering. In data-sparse scenarios, it generates reasonable recommendations by optimizing objective functions, such as maximizing user satisfaction—to compensate for collaborative filtering's reliance on historical data.

(9) AMCSI: This method employs an attention mechanism to generate career planning recommendations. However, the attention mechanism requires calculating the similarity between all feature pairs, resulting in significantly increased computational costs when dealing with high-dimensional features.

(10) JFormer: This method designs a recommendation system based on the Transformer mechanism, but the Transformer's self-attention mechanism requires substantial computational resources and memory.

4.2 Analysis of parameter impact and comparison of aggregation methods

4.2.1 Model parameter impact analysis

Experiments are conducted using the three datasets TF, JD, and TREC. 70% of each dataset is used as a training set, with 30% serving as a test set. Evaluation metrics include recommendation accuracy (ACC) and AUC. Hyperparameters include the number of neighbor samples k and sampling depth h. The impact comparison of hyperparameters is shown in Fig. 6.

The number of neighbor samples k refers to the quantity of neighboring nodes around each node used for graph convolution operations within a triplet graph structure. The number of neighbor samples refers to the quantity of neighboring nodes sampled for each node during graph convolution operations. It defines the scope for feature aggregation and propagation at the node level. The sensitivity of various performance metrics to the parameter k is shown in Fig. 6(a) and Fig. 6(b). The results reveal a clear trend: while performance remains relatively stable with smaller k values, larger settings induce greater divergence in model behavior, as seen from the widening gaps between metric curves. This instability at higher k may be attributed to increased computational complexity and memory footprint. Overall, the optimal performance on the TF, JD, and TREC datasets is achieved at k values of 8, 4, and 4, respectively. Sampling depth h determines how deep the model expands outward from a node's neighbors during this

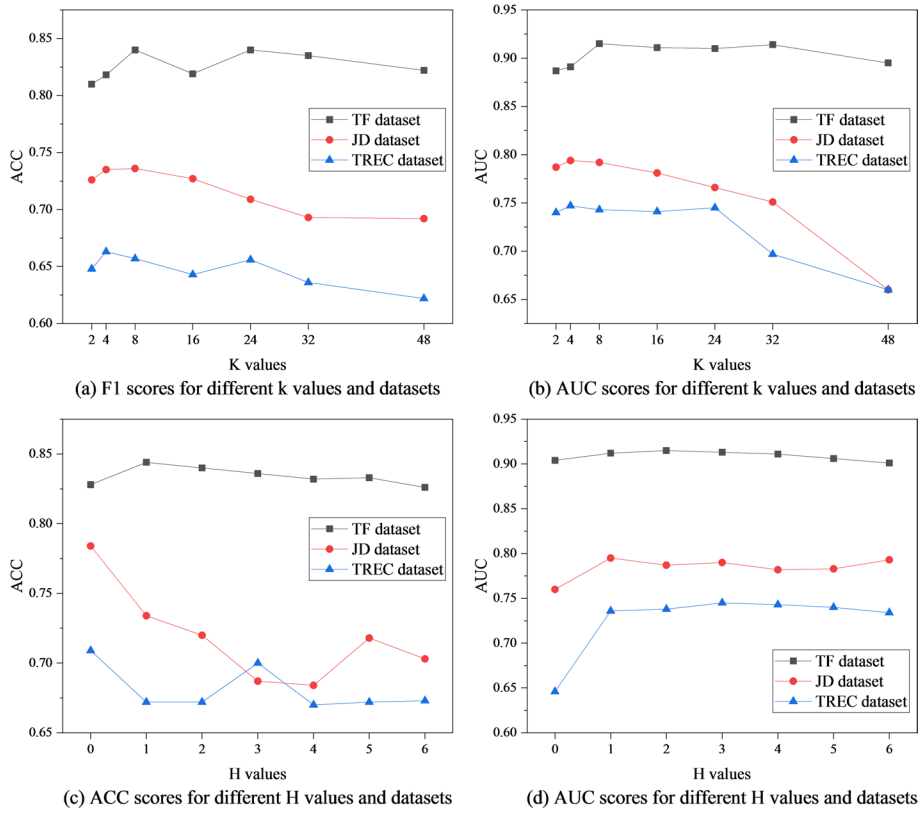


Fig. 6 Comparison of the effects of model hyperparameters

aggregation operation along the graph structure. A higher sampling depth means that the model will acquire information at more distant neighbor levels to capture broader semantic associations and structural features. From Fig. 6(c) and Fig. 6(d), it can be seen that as h gradually increases, KGIMCS shows significantly different performance across various datasets. Through comparison, we find that when KGIMCS achieves optimal performance on different datasets, the differences in h values become minimal afterward. According to the chart results, for TF, JD, and TREC, taking h values of {1, 1, 1} yields the best performance.

Adjusting the number of epochs in a model refers to the process of determining how many times the data should be run through during machine learning model training. The goal of epoch tuning is to find a point where the model performs well on the training set and also generalizes effectively on the validation set. If the model trains for too few epochs, it may fail to fully learn the patterns in the data, leading to underfitting. Conversely, too many epochs may cause the model to overlearn noise and minor details in the training data, resulting in overfitting. Gradually decreasing the learning rate during training helps the model make finer weight adjustments and avoids excessive oscillations in the gradient space. Adjusting the number of epochs is an empirical and experiment-driven process, determined by the specific model characteristics and the complexity of the dataset. Typically, more complex or noisy datasets may require more epochs, while simpler or smaller datasets may need fewer.

Figure 7 illustrates the relationship between recommendation accuracy metrics and training iterations for KGIMCS on the TF, JD, and TREC datasets. Initially, metric results steadily improve with increasing training iterations. On the TF dataset, ACC stabilizes around the 100th training iteration. On the JD dataset, ACC stabilizes around the 120th training epoch. On the TREC dataset, the KGIMCS model stabilizes after 200

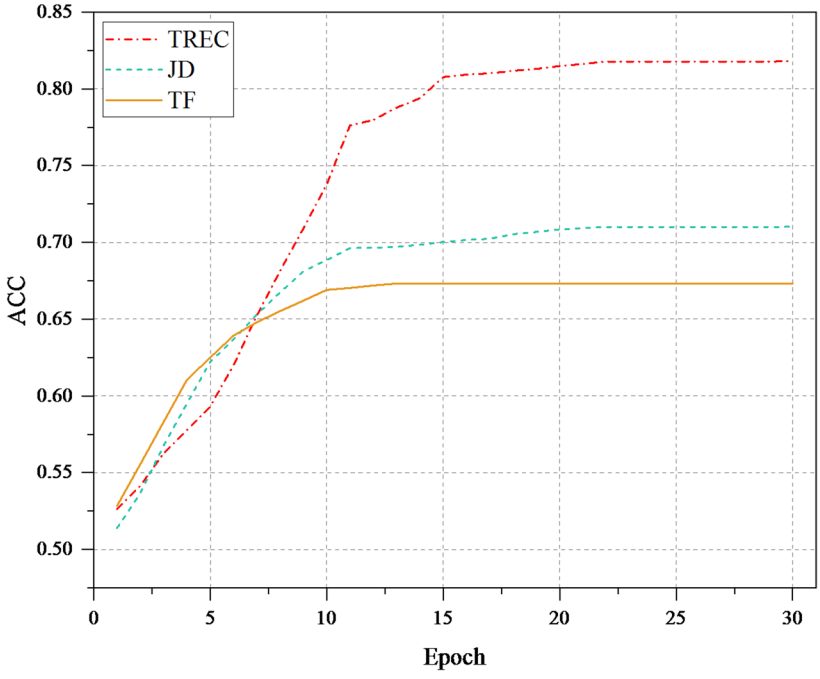


Fig. 7 Epoch parameter tuning across different datasets

training epochs. Therefore, the epoch training counts are set to 100, 120, and 200 for the TF, JD, and TREC datasets, respectively.

4.2.2 Comparison of different aggregation methods

In this study, node features differ across three different datasets; therefore, experiments are conducted separately for each aggregation method to select the best one for each dataset. From Table 5, it can be observed that Sum aggregator and Concat aggregator usually perform similarly on various datasets, while Neighbor aggregator's performance is significantly lower than those two. Evaluation based on AUC and ACC shows that the Sum aggregator delivers robust performance in most settings, particularly on the TF dataset. The Concat aggregator, however, holds a narrow advantage in certain instances, notably on the JD dataset and TREC dataset. The Neighbor aggregator appears to perform poorly across all datasets, regardless of AUC, ACC, or training time. These findings suggest that the choice of aggregator has a significant impact on model performance across different datasets; therefore, in subsequent experiments, this paper will use the Concat aggregator to aggregate neighboring nodes for the JD and TREC datasets, and the Sum aggregator for the TF dataset.

4.2.3 Analysis of knowledge graph completion performance

To validate the proposed word2vec+TransE knowledge completion method, this paper conducts link prediction experiments on the test set and evaluates performance through entity ranking. Specifically, for each test triplet, candidate entities replace the head or tail entity to be predicted, and the score for each candidate triplet is computed. Subsequently, all candidate entities are ranked in descending order of their scores, yielding an accurate ranking of candidate triplets. This paper employs two evaluation metrics: Hit Rate @K (Hit@K) and Mean Reciprocal Rank (MRR). The Hit@K metric represents the percentage of all factual triples ranked within the top K positions, where typical values for K are K= {1,10}. The MRR metric calculates the average reciprocal rank of the target entity among predicted candidate entities. Higher scores are assigned when the correct answer ranks higher, thereby measuring overall prediction quality. Higher values of MRR and Hit@K indicate stronger link prediction capabilities of the model.

As shown in Table 6, on TF, JD, and TREC datasets, our proposed word2vec+TransE knowledge graph completion method consistently achieves the best performance regardless of the dataset characteristics. This approach deeply integrates Word2Vec's unstructured text semantic mining capabilities with TransE's structured relationship modeling capabilities. It addresses the shortcomings of traditional knowledge graph completion methods in semantic understanding while compensating for the limitations of pure word embedding models in capturing explicit relationships between entities.

Table 5 Performance comparison of recommended aggregation methods

Model	Evaluation indicators	Dataset		
		TF	JD	TREC
Sum	AUC	0.9301	0.7784	0.7149
	ACC	0.8519	0.7157	0.6024
Concat	AUC	0.9154	0.7823	0.7546
	ACC	0.8457	0.7036	0.6157
Neighbor	AUC	0.9048	0.7452	0.5869
	ACC	0.8102	0.6755	0.5346

Table 6 Comparison of knowledge graph completion performance on TF, JD, and TREC datasets

Method	TF			JD			TREC		
	MRR	Hit@1	Hit@10	MRR	Hit@1	Hit@10	MRR	Hit@1	Hit@10
LFM	0.430	0.370	0.545	0.289	0.198	0.471	0.248	0.300	0.558
NGCF	0.482	0.399	0.606	0.247	0.158	0.428	0.440	0.410	0.510
KGMR	0.508	0.442	0.613	0.346	0.250	0.542	0.497	0.452	0.582
IHGCN	0.530	0.447	0.695	0.518	0.460	0.626	0.440	0.361	0.581
CMAKG	0.501	0.409	0.640	0.357	0.271	0.586	0.486	0.468	0.590
KGLRS	0.541	0.464	0.708	0.541	0.478	0.634	0.483	0.415	0.602
MVIDL	0.473	0.435	0.594	0.467	0.369	0.513	0.367	0.409	0.527
OURS	0.605	0.511	0.749	0.587	0.529	0.688	0.544	0.493	0.648

The proposed method effectively avoids the limitations of single features by comprehensively capturing semantic relationships within the global path scope. On the TF dataset, the MRR metric improved by 11.82%, Hits@1 increased by 10.12%, and Hits@10 rose by 5.79%. On the JD dataset, the MRR metric improved by 8.50%, Hits@1 increased by 10.66%, and Hits@10 rose by 8.51%. On the TREC dataset, the MRR metric improved by 9.45%, Hits@1 increased by 5.34%, and Hits@10 rose by 7.64%. In summary, the OURS method demonstrates strong relationship prediction capabilities and model generalization performance.

4.3 Comparative experiment analysis

4.3.1 Comparison of recommendation performance among models

In this subsection, we experimentally compare KGIMCS with benchmark models LFM, NGCF, KGMR, IHGCN, CMAKG, KGLRS, MVIDL, MACPF, AMCSI, and JFormer to validate the effectiveness of the proposed KGIMCS model, as shown in Fig. 8.

On the TF dataset, compared to other models, KGIMCS achieved AUC improvements of approximately 9%, 2%, 1.5%, 1%, 1.5%, 0.01%, 7.63%, 1.39%, 4.89%, and 2.17%; while its ACC values improved by 3.44%, 2.87%, 2.47%, 1.64%, 2.4%, 2.04%, 6.71%, 1.49%, 6.14%, and 3.15%, respectively. On the JD dataset, compared to the six models, KGIMCS achieved AUC improvements of 11.3%, 11.26%, 8.54%, 5.7%, 4.71%, 1.85%, 9.20%, 3.43%, 13.31%, and 4.03%, respectively. while its ACC improved by 13.62%, 11.57%, 8.44%, 4.8%, 3.52%, 2.26%, 10.67%, 3.50%, 13.14%, and 3.06%, respectively. On the TREC dataset, KGIMCS achieved AUC improvements ranging from 2.75% to 14.73% and ACC gains from 1.34% to 14%. These data indicate that KGIMCS performs well on the three datasets. In particular, both AUC and ACC show significant improvements of about 10% for KGIMCS over other models, especially in the JD dataset compared to other knowledge graph-based recommendation algorithms. This indicates that KGIMCS can more accurately predict user preferences regarding career planning knowledge, thereby improving the accuracy of the recommendation system. The increase in ACC further demonstrates that KGIMCS more effectively represents the characteristics of student-career knowledge interactions, directly enhancing personalization. This shows the model's superior capability to precisely identify patterns in student attention and career knowledge, which in turn yields more accurate recommendations.

In addition, this paper also uses Precision, Recall, F1 and NDCG to analyze the recommendation performance of different models, and the results are shown in Table 7. F1 is the harmonic mean of precision and recall, and it is a key metric for evaluating recommendation performance. The F1 values of KGIMCS on the TF, JD and TREC datasets

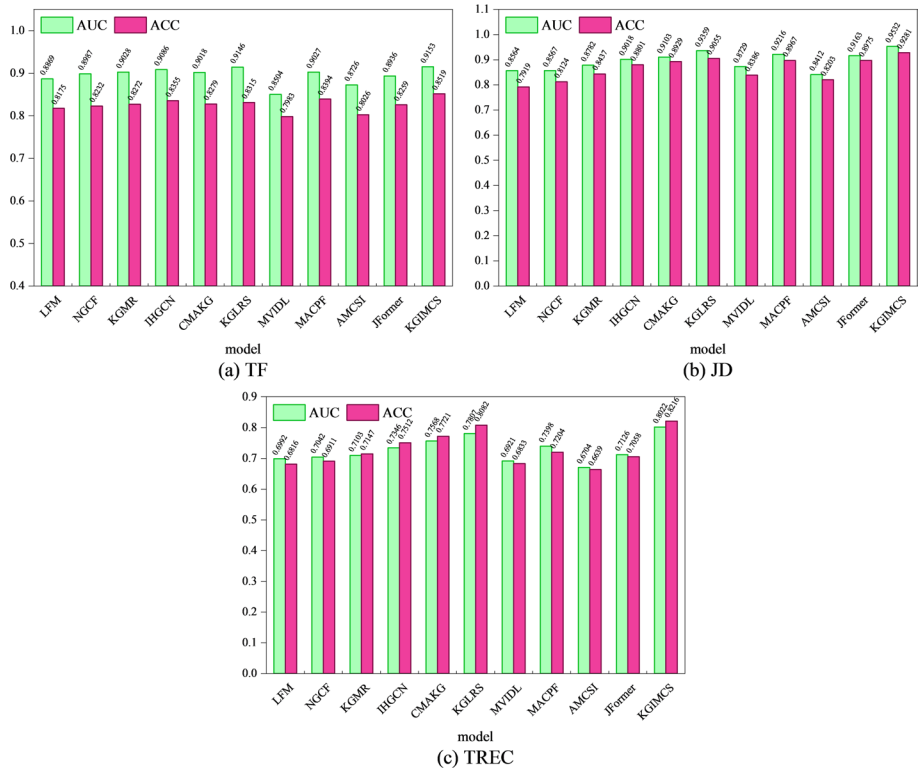


Fig. 8 Comparison of recommended performance indicators of each model

were 0.8621, 0.9383 and 0.8298 respectively, which were at least 1.64%, 4.9% and 2.21% higher than those of the other six models. NDCG is an important indicator for measuring the quality of recommendations. The NDCG of KGIMCS on the TF, JD and TREC datasets were 0.6293, 0.8039 and 0.5214 respectively, which were at least 10.36, 2.88 and 9.75% higher than those of the benchmark model. Through the analysis of the recommendation performance indicators of different models, it can be known that KGIMCS has a relatively high recommendation accuracy and can recommend relatively accurate career planning knowledge for college students.

4.3.2 Statistical significance test

After 100 independent runs, the mean and standard deviation of ACC and AUC for the college student career planning recommendation method on the TREC dataset were obtained. Furthermore, the renowned Wilcoxon signed-rank test was employed to analyze the results, where "+", "−", and "≈" denote that the comparison model is significantly better than KGIMCS, significantly worse than KGIMCS, and statistically similar to KGIMCS, respectively. The ACC and AUC metrics alongside Wilcoxon rank-sum test results are presented in Table 8. It is evident that KGIMCS exhibits higher maximum, minimum, average, and standard deviation values for both ACC and AUC compared to the comparison models. This demonstrates KGIMCS's significant superiority in diversity and convergence over the other models. The test results confirm KGIMCS's outstanding performance in college student career planning recommendations.

Table 7 The recommendation effects of different models

Model	Evaluation indicators	Dataset		
		TF	JD	TREC
LFM	Precision	0.6934	0.7863	0.6209
	Recall	0.6627	0.7423	0.5817
	F1	0.6777	0.7637	0.6007
	NDCG	0.6209	0.6217	0.2946
NGCF	Precision	0.7389	0.8025	0.6891
	Recall	0.7063	0.7819	0.6287
	F1	0.7222	0.7921	0.6575
	NDCG	0.6174	0.6519	0.3271
KGMR	Precision	0.7751	0.8419	0.7153
	Recall	0.7524	0.8179	0.6591
	F1	0.7636	0.8297	0.6861
	NDCG	0.5748	0.6641	0.3691
IHGCN	Precision	0.8021	0.8702	0.7693
	Recall	0.7937	0.8097	0.7314
	F1	0.7979	0.8389	0.7499
	NDCG	0.5064	0.7051	0.4176
CMAKG	Precision	0.8281	0.8815	0.7851
	Recall	0.8157	0.8571	0.7791
	F1	0.8219	0.8691	0.7821
	NDCG	0.5331	0.7296	0.4563
KGLRS	Precision	0.8503	0.9026	0.8206
	Recall	0.8411	0.8763	0.7952
	F1	0.8457	0.8893	0.8077
	NDCG	0.5702	0.7814	0.4751
MVIDL	Precision	0.8129	0.7724	0.7007
	Recall	0.7834	0.7903	0.7115
	F1	0.7979	0.7812	0.7061
	NDCG	0.5269	0.6627	0.4023
MACPF	Precision	0.8139	0.8702	0.7839
	Recall	0.8206	0.8539	0.7504
	F1	0.8172	0.8620	0.7668
	NDCG	0.6019	0.7429	0.4836
AMCSI	Precision	0.7503	0.7905	0.7201
	Recall	0.7718	0.8139	0.7395
	F1	0.7609	0.8020	0.7297
	NDCG	0.5157	0.6893	0.4219
JFormer	Precision	0.8129	0.8406	0.7439
	Recall	0.7928	0.8316	0.7603
	F1	0.8027	0.8361	0.7520
	NDCG	0.5301	0.7025	0.4502
KGIMCS	Precision	0.8715	0.9425	0.8327
	Recall	0.8529	0.9341	0.8269
	F1	0.8621	0.9383	0.8298
	NDCG	0.6293	0.8039	0.5214

4.3.3 Real-time analysis of recommendations

The computational efficiency of each model was evaluated by comparing their per-recommendation time across all datasets, as summarized in Fig. 9. KGIMCS achieves inference times of 2.6 ms, 6.8 ms, and 2.0 ms on the TF, JD, and TREC datasets, representing the lowest latency among all models, surpassing the others by at least 1.1 ms, 1.0 ms, and 2.7 ms, respectively. This efficiency stems from KGIMCS's integration of GMF, an

Table 8 Statistical significance test results

Model	Mean value	Standard deviation	Maximum	Minimum	Significance test
LFM	ACC	0.6195	0.7051	0.6217	–
	AUC	0.6387	0.7144	0.6495	–
NGCF	ACC	0.6504	0.7396	0.6915	–
	AUC	0.6715	0.7415	0.7022	–
KGMR	ACC	0.6902	0.7566	0.7051	–
	AUC	0.7041	0.7422	0.7147	–
IHGCN	ACC	0.7184	0.7624	0.7217	–
	AUC	0.7351	0.7761	0.7405	–
CMAKG	ACC	0.7501	0.7901	0.7504	–
	AUC	0.7694	0.7814	0.7691	–
KGLRS	ACC	0.7852	0.8123	0.7736	–
	AUC	0.7417	0.7933	0.7712	–
MVIDL	ACC	0.7203	0.7602	0.6847	–
	AUC	0.7811	0.8125	0.7033	–
MACPF	ACC	0.7552	0.7714	0.7136	–
	AUC	0.7702	0.7902	0.7439	–
AMCSI	ACC	0.7519	0.7825	0.7126	–
	AUC	0.7305	0.7627	0.7071	–
JFormer	ACC	0.7726	0.7935	0.7039	–
	AUC	0.7153	0.7625	0.6893	–
KGIMCS	ACC	0.8169	0.8505	0.7936	
	AUC	0.8726	0.8812	0.8021	

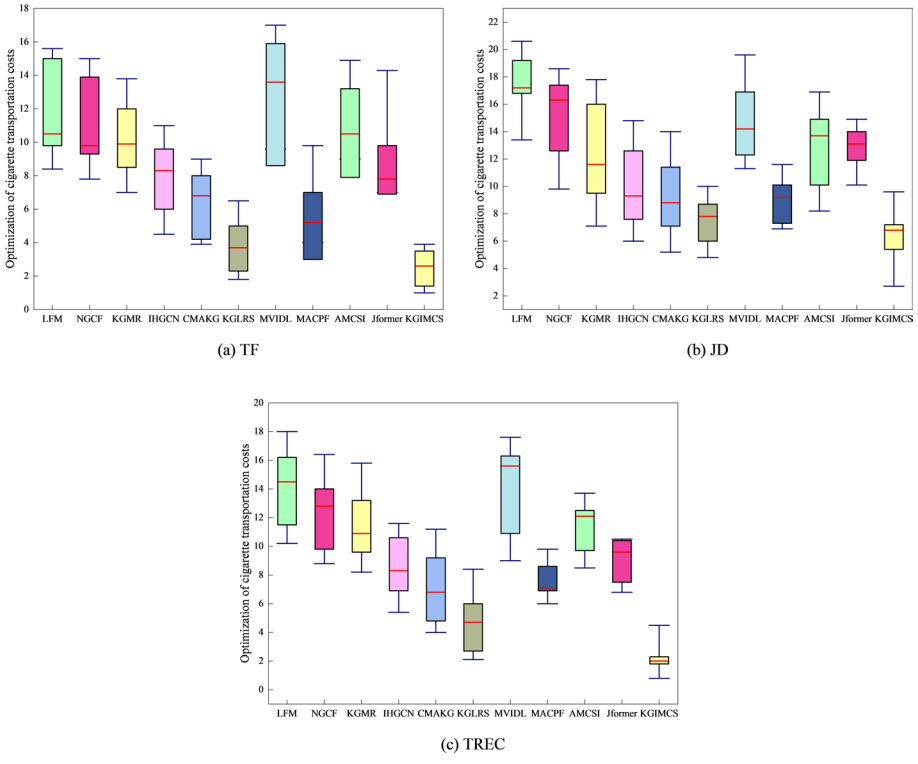


Fig. 9 Real-time comparison of recommendation models

enhanced matrix factorization algorithm, with a knowledge graph into a unified framework. The synergy between these components enhances computational performance, enabling superior real-time response capabilities.

4.3.4 Model complexity analysis

In addition to analyzing the real-time performance of the aforementioned models, this paper also compares their spatio-temporal complexity, inference time, memory usage, and training time. The comparison results are shown in Table 9, where N_e, N_r, and N_t represent the number of entities, relations, and attributes, respectively. The proposed KGIMCS model outperforms the comparison models in terms of spatio-temporal complexity, inference time, memory usage, and training time.

KGIMCS can be decomposed into three modules: knowledge graph feature extraction, GMF explicit preference modeling, and GCN implicit relationship inference. Each module is independently optimized to avoid the accumulation of high complexity inherent in a single model. The KGIMCS model employs GMF for matrix factorization, modeling user-item interactions as the product of low-rank matrices. This approach achieves significantly lower spatio-temporal computational complexity compared to the layer-by-layer nonlinear transformations of deep learning models. In career planning recommendations, GMF rapidly captures explicit preferences between users and occupations. Additionally, GCN achieves message passing by aggregating neighboring node features, resulting in substantially lower computational complexity than alternative models.

4.3.5 Cold start scenario comparison experiment

To validate the effectiveness of KGIMCS in cold-start scenarios with limited user interaction data, this paper conducts experiments using interaction data from 200 to 300 randomly selected students from the TREC dataset. To prevent data leakage and more realistically simulate the state of new users entering the system, a rigorous sampling method was designed. The sampling criterion ensures that the selected students do not appear in the training set (i.e., cold-start users). Specifically, prior to splitting the dataset into training and testing sets, we randomly selected 200 and 300 users from the TREC dataset as cold-start users. All interaction records involving these users were removed from the original training set and evaluated as a separate testing group.

For the goal of simulating the initial behavior of new users within the system, this paper randomly selects two vocational education courses for each cold-start user as their initial interactions. First, for each cold-start user, two items are randomly sampled from the original training set to form their interaction records. Second, these interaction records are added to the test set and used solely as model inputs without affecting the data in the training phase. In summary, to prevent cold-start user information from interfering with model training, this paper employs the following strict training/testing separation mechanism.

(1) Interaction records from all non-cold-start users constitute the training set.
(2) Cold-start user interactions are used solely for model evaluation during testing and do not participate in any parameter updates or loss calculations during model training.
(3) The raw interactions of cold-start users are removed and used solely as target predictions during evaluation.

Table 9 The complexity analysis results of the model

Model	Time complexity	Space complexity	Reasoning time (ms)	Memory usage (%)	Training time (min)
LFM	$O((N_e + N_r + N_t) \times d^2)$	$O((N_e + N_r) \times d^2)$	139	79	143
NGCF	$O((N_e + N_r)d + N_t \times d^2)$	$O(N_e \times d + N_r \times d^2)$	120	75	117
KGMR	$O(N_e + N_r \times d + N_t \times d^2)$	$O(N_e \times d^2 + N_r \times d)$	95	68	96
IHGCN	$O(N_e \times d^2 + N_r \times d + N_t \times d)$	$O(N_e \times d^2 + N_r)$	79	62	72
CMAKG	$O((N_e + N_r + N_t) \times d)$	$O((N_e + N_r) \times d)$	71	46	38
KGLRS	$O(N_e \times d + N_r \times d + N_t)$	$O(N_e \times d + N_r)$	42	51	57
MVIDL	$O((N_e + N_r)d + N_t \times d^2)$	$O(N_e \times d^2 + N_r)$	132	74	139
MACPF	$O((N_e + N_r + N_t) \times d)$	$O((N_e \times d + N_r \times d^2)$	55	58	69
AMCSI	$O((N_e + N_t) \times d + N_r \times d^2)$	$O(N_e + N_r \times d)$	86	69	126
JFormer	$O((N_e \times d + (N_r + N_t) \times d^2)$	$O((N_e + N_r) \times d^2)$	117	78	75
KGIMCS	$O(N_e + N_r + N_t)$	$O(N_e + N_r)$	25	37	32

(4) Historical information from cold-start users is completely excluded from the model training process.
(5) During the testing phase, recommendation predictions are made exclusively for cold-start users (based on their two initial interactions).

Figures 10 and 11 respectively illustrate the performance comparison between KGIMCS and ten other models in terms of metrics such as ACC and AUC under scenarios with 200 and 300 cold-start users. Experimental results demonstrate that KGIMCS achieves significantly higher ACC and AUC scores than baseline methods in recommendation tasks for cold-start users. The primary reason for KGIMCS's outstanding performance lies in its ability to leverage additional semantic information from knowledge graphs (such as user-course and course-knowledge point relationships) to effectively enhance user embedding representations even with limited interactions. Furthermore, by integrating knowledge graphs with matrix factorization, KGIMCS improves the model's capability to handle sparse data.

4.4 Scalability analysis of the model

To ensure the scientific rigor and reliability of the scalability experiment, the expanded dataset employs a hybrid construction strategy combining real data as the foundation with synthetic data to augment the volume. The foundational dataset utilizes TF, incorporating generative adversarial network (GAN) models to generate synthetic data. First, feature extraction is performed on real data to construct marginal and joint distribution models for entities such as users, occupations, and skills, clarifying the probabilistic patterns of core relationships like profession-skill-occupation. Subsequently, the model learns the entity association patterns of the real knowledge graph to generate virtual entities and relationships conforming to identical distribution characteristics. Finally, logical rules filter out unreasonable data to ensure validity. Each expansion level (5×, 10×, 20×, 50×, 100×) is scaled proportionally to the entities in the baseline dataset, with consistent relationship types and proportions. Triple-layer quality validation is implemented: KS test verifies consistency between synthetic and real data distributions (similarity $\geq 95\%$); logical rules filter conflicting data (conflict rate $\leq 0.3\%$); integrity checks control core attribute missing rate $\leq 0.5\%$, ensuring expanded data quality.

Scalability analysis verifies a model's adaptability and improvement potential under varying conditions. This paper selects knowledge graph scale (number of entities/relationships), user data volume, and occupation count as independent variables, while using offline training time, online inference time, peak memory consumption, and recommendation accuracy (ACC) as dependent variables. The impact of independent variable scaling on model performance is shown in Table 10.

Based on offline training times, as the dataset scale expands, training duration increases from 2.3 min to 88.5 min, exhibiting near-linear growth overall. This is because the model's training complexity scales linearly with the number of entities and relationships, while the batch processing mechanism effectively mitigates exponential growth trends. Online inference time increased from 4.6 ms to 98.6 ms. Before data scale expansion, the growth was relatively gradual. After expansion, the growth rate accelerated but remained within 100ms, meeting real-time recommendation requirements. The latency is generally acceptable to users, with ≤ 500 ms being tolerable. This is attributable to the complementary features of GMF and GCN, coupled with GPU acceleration for model

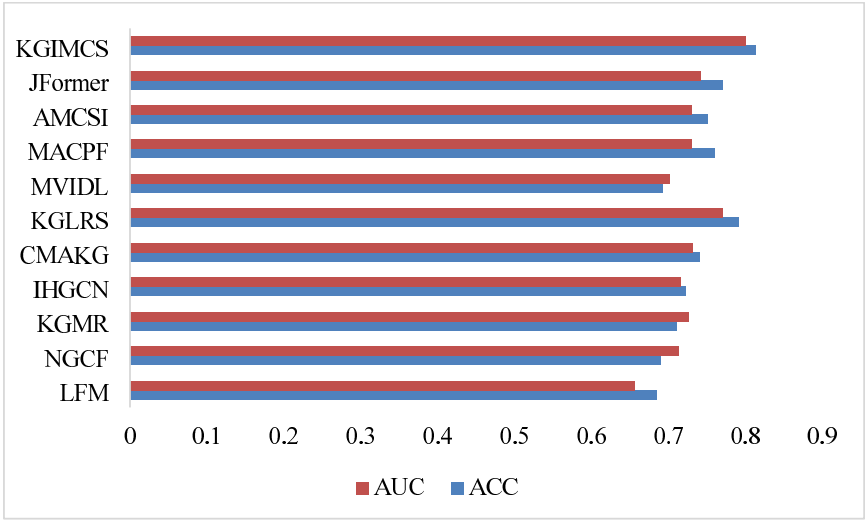


Fig. 10 Performance comparison of scenario models for 300 students

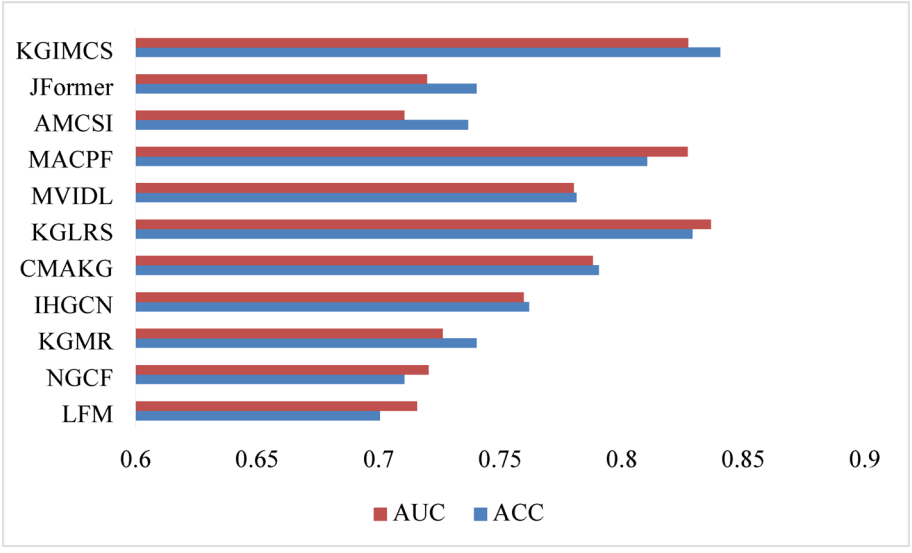


Fig. 11 Performance comparison of scenario models for 200 students

Table 10 The impact of independent variable scaling on model performance

Number of entities (ten thousand)	Relation number (ten thousand)	Number of users (ten thousand)	Number of occupations (ten thousand)	Offline training time (min)	Online inference time (ms)	Peak memory usage (GB)	ACC (%)
0.5	2	0.1	0.05	2.3	4.6	0.7	81.9
2.5	10	0.5	0.25	15.7	18.2	5.3	82.6
5	20	1	0.5	28.9	35.4	7.8	82.4
10	40	2	1	32.4	44.7	10.5	83.7
25	100	5	2.5	56.8	76.3	16.2	81.5
50	200	10	5	88.5	98.6	20.8	82.3

inference. Memory consumption exhibits a linear positive correlation with data scale, peaking at 20.8 GB without exceeding hardware memory limits. This demonstrates the scalability of the data storage and loading mechanisms. The model's recommendation accuracy consistently remained stable between 81.5% and 83.7%, with fluctuations ≤ 3%. This demonstrates the method's strong stability and robustness in recommendation logic when scaling data size, without any decline in matching accuracy due to increased data volume.

4.5 Ablation experiment

To further validate the importance of each component in the proposed model, this paper conducted ablation studies on the components of the recommended KGIMCS model. The GCN knowledge graph aggregation module was replaced with a basic GNN aggregation, denoted as KGIMCS/GCN. The GMF decomposition module was removed and replaced with a traditional MF module, denoted as KGIMCS/GMF. The weighted fusion module was changed to concatenation fusion, denoted as KGIMCS/SP. The knowledge graph module will be removed and designated as KGIMCS/KG. The ablation experiment results of different components on the TREC dataset are shown in Table 11. KGIMCS achieved an ACC of 0.8906 and an AUC of 0.9508, 0.9508, representing improvements of 8.43% and 9.89% over KGIMCS/GCN, 3.38% and 3.4% over KGIMCS/GMF, 6.94% and 5.41% over KGIMCS/SP, and 13.67% and 14.72% over KGIMCS/KG.

The results above demonstrate that the KGIMCS model, which integrates all components, achieves the best recommendation performance. For the aforementioned experimental results, this paper provides the following analysis of the reasons.

(1) The incorporation of knowledge graphs significantly enhances the recommendation performance of the KGIMCS model. Knowledge graphs enrich recommendations with detailed information and semantic connections. By integrating entity and relationship information from knowledge graphs with interaction data between college students and career planning knowledge, the KGIMCS algorithm gains a deeper understanding of students' interests and the unique characteristics of career planning knowledge, thereby improving the accuracy of the recommendation system.

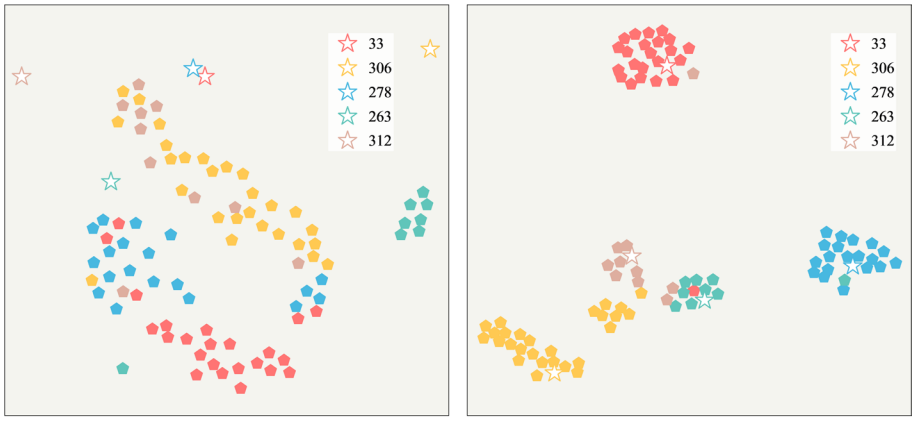
(2) Integrating GMF with the knowledge graph enables more efficient utilization of both algorithms' strengths, thereby boosting the recommendation system's performance. Furthermore, the combination of KGIMCS and GMF effectively addresses data sparsity and cold-start challenges, broadening the scope of recommendations while significantly improving their quality.

4.6 Explainability analysis

To more intuitively demonstrate the interpretability of KGIMCS in recommendation performance, this paper employs the visualization tool t-SNE to visualize the feature

Table 11 Statistical significance test results

Model	ACC	AUC
KGIMCS/GCN	0.8063	0.8519
KGIMCS/GMF	0.8568	0.9168
KGIMCS/SP	0.8212	0.8967
KGIMCS/KG	0.7539	0.8036
KGIMCS	0.8906	0.9508



(a) Visual analysis outcome of KGLRS (b) Visual analysis outcome of KGIMCS

Fig. 12 Results of feature visualization analysis

vectors of university students and career planning knowledge nodes. Five university students and their browsed career planning knowledge were randomly selected from the TREC dataset. Subsequently, the feature vectors learned by the benchmark model KGLRS and KGIMCS for university students and career planning knowledge were visualized separately. The visualization results are shown in Fig. 12, where content of the same color indicates access by the same university student.

By examining these two diagrams, it becomes apparent that the distance between career planning knowledge clicked by the same college student tends to be relatively close, whereas the distance between career planning knowledge clicked by different college students tends to be relatively far apart. Each student and their clicked content can be regarded as a cluster. Compared to KGLRS, KGIMCS enables tighter cohesion within clusters while promoting greater dispersion between clusters. This demonstrates the KGIMCS model's superiority in modeling user interest preferences.

5 Conclusion

To address the issue of low accuracy in current recommendation methods under information overload conditions, this paper proposes a knowledge graph-integrated career planning recommendation method for college students. First, data related to college students' career planning is collected, and structured information formed during natural language processing is extracted to construct a knowledge graph for college student career planning. Based on this, using triplet information from the knowledge graph, a corresponding neighborhood is established for each career planning knowledge entity through intermediate relationships. Subsequently, matrix factorization and GCN are employed to aggregate triplet information, generating corresponding embedded representations for both college students and career planning knowledge. Finally, the two types of embeddings are fused through a concatenation approach and computed to output the final recommendation results. Experimental results demonstrate that the proposed model achieves recommendation accuracy rates of 85.19%, 92.81%, and 82.16% on three real-world datasets, outperforming baseline models and validating the effectiveness of this approach. Although the proposed model achieved satisfactory results in experiments, it still has limitations. Future research will incorporate information from multiple professional domains to uncover latent connections and identify career

planning expertise that may interest college students. By integrating information from additional fields, the recommendation system can access more potential features, thereby enhancing the precision of the recommendation algorithm.

Although the proposed model achieved good results in the experiment. However, there are still deficiencies. In the future, improvements are planned to be made in the following directions.

(1) Cross-disciplinary recommendation. College students' interests are often not limited to a certain field. Therefore, in the future career planning knowledge recommendation work, information from other fields can be introduced, and potential correlations can be utilized to discover non-obvious knowledge that college students may be interested in. By introducing information from more fields, more possible features can be provided for the recommendation system, improving the precision and accuracy of the recommendation algorithm.

(2) Lightweighting of graph convolution technology. To address the inevitable over-smoothing issue in traditional GCN technology, this paper has always focused on optimizing the structure of the model's input data, but has overlooked the significance of optimizing the GCN technology itself. The complexity of the algorithm can be reduced by simplifying the model through optimizing the GCN network.

Acknowledgements
Not applicable.

Author contributions
Xiaojun Xu Statistical Analysis & Manuscript Preparation; Hui Gao Data Collection & Manuscript Preparation.

Funding
This work is supported by the Achievements of Yunnan Career Counseling Characteristic Studio Project and Science Research Fund of Yunnan Provincial Department of Education (Teacher Category).

Data availability
The datasets used and/or analysed during the current study available from the corresponding author on reasonable request.

Declarations

Ethics approval and consent to participate
This study did not require ethical approval as it did not involve human or animal subjects.

Consent for publication
Not applicable. This study does not contain any individual data requiring consent for publication.

Competing interests
The authors declare no competing interests.

Received: 20 October 2025 / Accepted: 5 February 2026
Published online: 29 March 2026

References

1. Amin S, Uddin MI, Alarood AA, Mashwani WK, Alzahrani AO, Alzahrani HA. An adaptable and personalized framework for top-N course recommendations in online learning. Sci Rep. 2024;14(1):103–12. https://doi.org/10.1038/s41598-024-56497-1.
2. Ashrafi S, Majidi B, Akhtarkavan E, Hajiagha SHR. Efficient resume-based re-education for career recommendation in rapidly evolving job markets. IEEE Access. 2023;11:124350–67. https://doi.org/10.1109/ACCESS.2023.3329576.
3. Cha S, Li H. Design and implementation of college students' employment guidance system based on knowledge graph. Front Soc Sci Technol. 2023;5(15):75–91. https://doi.org/10.25236/FSST.2023.051514.
4. Cheng Y. Development of a career planning assistance platform for college students combining fuzzy logic and deep learning. J Adv Comput Intell Intell Inf. 2025;29(3):445–55. https://doi.org/10.20965/jaciii.2025.p0445.
5. Desnelita Y, Susanti W. The development of career path recommendation expert system (cpres) model in higher education. Turkish J Comput Math Educ. 2021;12(14):4151–7.
6. Ebisu T, Ichise R. Generalized translation-based embedding of knowledge graph. IEEE Trans Knowl Data Eng. 2019;32(5):941–51. https://doi.org/10.1109/TKDE.2019.2893920.

7. Floris M, Paganin G, Guglielmi D, Mazzetti G. Motivation is not enough: how career planning and effort regulation predict academic achievement. Curr Psychol. 2024;43(10):9280–9. https://doi.org/10.1007/s12144-023-05070-6.
8. Giabelli A, Malandri L, Mercorio F, Mezzanzanica M, Seveso A. Skills2Job: A recommender system that encodes job offer embeddings on graph databases. Appl Soft Comput. 2021;10(4):107–19. https://doi.org/10.1016/j.asoc.2020.107049.
9. Gu X. Effect of deep learning on college students' career planning. Math Probl Eng. 2022;10(1):57–73. https://doi.org/10.1155/2022/1573635.
10. Guo J, Qi L. Visualization research of college students' career planning paths integrating deep learning and big data. Math Probl Eng. 2022;4(1):60–78. https://doi.org/10.1155/2022/6006968.
11. Han L, Zhao Z, Li H, Deng M, Zhou L. Design of online learning resource recommendation platform based on college students' career planning. Int J Social Sci Educ Res. 2022;5(1):272–7.
12. Huang Y. Individualized career planning intelligent teaching system for college students based on adaptive artificial intelligence. J Comput Methods Sci Eng. 2025;14(2):27–35. https://doi.org/10.1177/14727978251315731.
13. Isaac J, Bai VMA. Improving academic performance and career mobility through hybrid clustered graph neural networks. Trans Emerg Telecommunications Technol. 2025;36(7):70–84. https://doi.org/10.1002/ett.70190.
14. Johnson SJ, Murty MR, Navakanth I. A detailed review on word embedding techniques with emphasis on word2vec. Multimedia Tools Appl. 2024;83(13):37979–8007. https://doi.org/10.1007/s11042-023-17007-z.
15. Lahoud C, Moussa S, Obeid C, Khoury HE, Champin P-A. A comparative analysis of different recommender systems for university major and career domain guidance. Educ Inform Technol. 2023;28(7):8733–59. https://doi.org/10.1007/s10639-022-11541-3.
16. Lee D, Kim M, Na I. Artificial intelligence based career matching. J Intell Fuzzy Syst. 2018;35(6):6061–70. https://doi.org/10.3233/JIFS-169846.
17. Miwa M, Takahashi H. Knowledge acquisition and modification during students' exploratory web search processes for career planning. Inform Res. 2008;13(4):13–4.
18. Ramazanova V, Sambetbayeva M, Serikbayeva S, Sadirmekova Z, Yerimbetova A. Development of a knowledge graph-based model for recommending MOOCs to supplement university educational programs in line with employer requirements. IEEE Access. 2024;12:193313–31. https://doi.org/10.1109/ACCESS.2024.3519263.
19. Ren Y. Prediction of career tendency and personalized suggestions for college students under cluster analysis. J Comput Methods Sci Eng. 2025;25(2):1951–62. https://doi.org/10.1177/14727978241307474.
20. Rosenberger J, Wolfrum L, Weinzierl S, Kraus M, Zschech P. CareerBERT: matching resumes to ESCO jobs in a shared embedding space for generic job recommendations. Expert Syst Appl. 2025;275:127–43. https://doi.org/10.1016/j.eswa.2025.127043.
21. Shi Z, Wu Z, Zhang Z, Chen Y, Liu X. Learning path planning algorithm based on career goals and artificial intelligence. Int J Emerg Technol Learn (IJET). 2022;17(10):256–72. https://doi.org/10.3991/ijet.v17i10.28455.
22. Siswipraptini PC, Warnars HLHS, Ramadhan A, Budiharto W. Personalized career-path recommendation model for information technology students in Indonesia. IEEE Access. 2024;12:49092–105. https://doi.org/10.1109/ACCESS.2024.3381032.
23. Tatchum GW, Nzeko'o AJN, Nguele TM, Djam XY. A general graph learning based framework for job offer recommender system. Eng Appl Artif Intell. 2025;16(2):17–38. https://doi.org/10.1016/j.engappai.2025.111713.
24. Verma P, Sood SK, Kalra S. Student career path recommendation in engineering stream based on three-dimensional model. Comput Appl Eng Educ. 2017;25(4):578–93. https://doi.org/10.1002/cae.21822.
25. Wang H, Yang W, Li J, Ou J, Song Y, Chen Y. An improved heterogeneous graph convolutional network for job recommendation. Eng Appl Artif Intell. 2023;126:81–96. https://doi.org/10.1016/j.engappai.2023.107147.
26. Wang J. Vocational college employment training and career planning model design based on improved collaborative filtering. Int J Comput Syst Eng. 2025;9(13):1–10. https://doi.org/10.1504/IJCSYSE.2025.148029.
27. Wang J, Xie H, Wang FL, Lee L-K, Au OTS. Top-N personalized recommendation with graph neural networks in MOOCs. Computers Education: Artif Intell. 2021;2:95–117. https://doi.org/10.1016/j.caeai.2021.100010.
28. Widayanti R, Chakim MHR, Lukita C, Rahardja U, Lutfiani N. Improving recommender systems using hybrid techniques of collaborative filtering and content-based filtering. J Appl Data Sci. 2023;4(3):289–302.
29. Yang Y, Zhang C, Song X, Dong Z, Zhu H, Li W. Contextualized knowledge graph embedding for explainable talent training course recommendation. ACM Trans Inform Syst. 2023;42(2):1–27. https://doi.org/10.1145/3597022
30. Zha R, Qin C, Zhang L, Shen D, Xu T, Zhu H, Chen E. Career mobility analysis with uncertainty-aware graph autoencoders: A job title transition perspective. IEEE Trans Comput Social Syst. 2023;11(1):1205–15. https://doi.org/10.1109/TCSS.2023.3239038.
31. Zhang N, Wu C. Application of deep learning in career planning and entrepreneurship of college students. J Comput Methods Sci Eng. 2024;24(5):2927–42. https://doi.org/10.1155/2022/8073232.
32. Zhang X, Liu S, Wang H. Personalized learning path recommendation for e-learning based on knowledge graph and graph convolutional network. Int J Software Eng Knowl Eng. 2023;33(1):109–31. https://doi.org/10.1142/S0218194022500681.
33. Zhang Y. Path of career planning and employment strategy based on deep learning in the information age. PLoS ONE. 2024;19(10):45–59. https://doi.org/10.1371/journal.pone.0308654.
34. Zhang Z, Luo G, Zhang J. A Multi-User collaborative recommendation mechanism for career planning in online learning over edge networks. Internet Technol Lett. 2025;8(5):e70126. https://doi.org/10.1002/itl2.70126.
35. Zhou Y, Zhang Y, Yu S, Liu N. Research on the design of an AI career path recommendation system based on MBTI from a Cross-Cultural perspective. Artif Intell Technol Res. 2024;2(1):51–63. https://doi.org/10.18686/aitr.v2i1.3857.
36. Zhu G, Chen Y, Wang S. Graph-community-enabled personalized course-job recommendations with cross-domain data integration. Sustainability. 2022;14(12):39–52. https://doi.org/10.3390/su14127439.
37. Guan Y, Wei Q, Chen G. Deep learning based personalized recommendation with multi-view information integration. Decis Support Syst. 2019;118:58–69. https://doi.org/10.1016/j.dss.2019.01.003.
38. Nithya B, Evangelin Geetha D, Kumar M. Metaheuristic-Assisted contextual Post-Filtering method for event recommendation system. Int J Image Graphics. 2025;25(5):255–78. https://doi.org/10.1142/S0219467825500433.
39. Zhang H, Zheng Z. Application and analysis of artificial intelligence in college students' career planning and employment and entrepreneurship information recommendation. Secur Communication Networks. 2022;4(1):80–97. https://doi.org/10.1155/2022/8073232.

40. Guan Z, Yang J-Q, Yang Y, Zhu H, Li W, Xiong H. JobFormer: skill-aware job recommendation with semantic-enhanced transformer. ACM Trans Knowl Discovery Data. 2024;19(1):1–20. https://doi.org/10.1145/3701735.

Publisher's note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.